Hotel Reservation & Booking System Final Project Proposal

Project Title: “SuiteSpot“
Group 1
C3A
Group Members:
Cortez, Cedric
Elpedes, Dustin
Maliñana, Johnbel

I. Introduction

A. Project Overview

The Hotel Reservation & Booking System is a full-stack MERN web application that digitizes hotel front-desk and guest operations. It is adapted from an existing Java console prototype (file-based storage, hardcoded staff credentials) into a modern, database-backed, browser-accessible platform. The system manages an 80-room hotel spanning 8 floors (10 rooms per floor, split across four room types — Single, Double, King, and Suite) and covers the full booking lifecycle: guest registration, room search and booking by date range, staff check-in/check-out, and administrative management of rooms, bookings, and staff accounts.

B. Objective

- Build a complete MERN stack application (React, Node.js, Express.js, MongoDB/Mongoose) that satisfies every course requirement: full CRUD, user authentication, role-based authorization, and both frontend and backend validation.
- Translate and extend the existing Java console Hotel Reservation System into a multi-user, real-time, availability-aware web application.
- Implement three-tier role-based access control (Guest, Receptionist, Manager) with clearly separated permissions.
- Track true room availability using individually numbered rooms and check-in/check-out date ranges, rather than the Java version's fixed quantity counters.
- Add practical guest-facing features beyond the original scope: filtering rooms by date, price, and capacity; room photos; and printable booking receipts.

II. System Features and Functionalities and

Create

- Guest self-registers an account and creates room bookings (dates, guest count, meal add-ons, payment).
- Manager adds new room records and creates Receptionist staff accounts.

Read

- Guest browses and filters available rooms by date range, price, and capacity; views own booking history and printable receipts.
- Receptionist and Manager view all bookings, filterable by date and payment status.
- Manager views all registered users.

Update

- Guest updates their own profile information.
- Manager edits room details (price, capacity, photo, status) and can deactivate staff accounts.
- Receptionist updates a booking's status (check-in, check-out).

Delete

- Guest cancels their own upcoming (not yet checked-in) booking.
- Manager deletes room records and deletes user accounts (guest or staff).

III. Roles

Guest (self-registered)

- Browse and filter rooms by date range, price, and capacity
- View room details and photos
- Book a room: select dates, guest count, meal add-ons, and full or partial (30%) payment
- View own bookings and a printable receipt
- Cancel an own upcoming booking
- Edit own profile

Receptionist (account created by the Manager, not self-registered)

- View all bookings; filter by date or payment status
- Check guests in and out
- No access to room management or user administration

Manager (Admin, top-level)

- Full CRUD on individual room records (add, edit, delete)
- View and cancel any booking
- View all registered users
- Create and deactivate Receptionist accounts
- Delete guest or staff accounts

IV. Technology Implementation Plan

A. Technology Stack

- **Frontend:** React (Vite), Tailwind CSS, React Router, Context API (auth state), Axios
- **Backend:** Node.js, Express.js (RESTful API)
- **Database:** MongoDB with Mongoose (ODM)

- **Authentication:** JSON Web Tokens (JWT) stored in an httpOnly cookie, bcrypt password hashing, role-based middleware
- **Receipts:** Print-friendly browser view (`window.print()`) rather than a PDF-generation library
- **Room Photos:** Admin-supplied image URLs (no file-upload service required)

B. Data Models (Mongoose Schemas)

User

Field	Type	Notes
name	String	required
email	String	required, unique
passwordHash	String	required
role	String	enum: guest, receptionist, manager
contact	String	
address	String	
isActive	Boolean	default true

Room

Field	Type	Notes
roomNumber	String	unique, e.g. "101"
floor	Number	1–8
type	String	enum: Single, Double, King, Suite
pricePerNight	Number	
maxOccupancy	Number	
photoUrl	String	
status	String	enum: available, maintenance

Booking

Field	Type	Notes
transactionId	String	unique, generated

Field	Type	Notes
guest	ObjectId (ref: User)	
room	ObjectId (ref: Room)	
checkIn	Date	
checkOut	Date	
guests	Number	
mealAddons	String	none / lunch / dinner / both
totalAmount	Number	
paidAmount	Number	
balance	Number	
paymentStatus	String	enum: full, partial
status	String	enum: confirmed, checked-in, completed, cancelled

C. API Endpoints (RESTful Routes)

Method	Endpoint	Description	Access
POST	/api/auth/register	Register a new guest account	Public
POST	/api/auth/login	Log in, issue JWT cookie	Public
POST	/api/auth/logout	Clear auth cookie	Authenticated
GET	/api/auth/me	Get current logged-in user	Authenticated
GET	/api/rooms	List/filter available rooms (date, price, capacity)	Public
GET	/api/rooms/:id	Get one room's details	Public
POST	/api/rooms	Add a new room	Manager
PUT	/api/rooms/:id	Edit room details	Manager
DELETE	/api/rooms/:id	Delete a room	Manager

Method	Endpoint	Description	Access
POST	/api/bookings	Create a booking	Guest
GET	/api/bookings/mine	View own bookings	Guest
GET	/api/bookings	View all bookings (filterable)	Receptionist, Manager
PATCH	/api/bookings/:id/checkin	Mark a booking checked-in	Receptionist, Manager
PATCH	/api/bookings/:id/checkout	Mark a booking checked-out/completed	Receptionist, Manager
PATCH	/api/bookings/:id/cancel	Cancel own booking (guest) or any booking (manager)	Guest (own), Manager (any)
GET	/api/users	List all users	Manager
POST	/api/users/receptionist	Create a Receptionist account	Manager
PATCH	/api/users/:id/deactivate	Deactivate a user account	Manager
DELETE	/api/users/:id	Delete a user account	Manager